Developer Documentation

Project: SongDB Music Library

Author: Mohamed Swesa

Neptun Code: D044RQ

Date: December 1, 2025

---

1. Explanation of Solution

Overview

The **SongDB** program is a C90 console application designed to manage a music library. The core solution relies on an **in-memory Singly Linked List** to store song data. This approach allows for dynamic growth of the database without fixed array limits.

Data Structures

The primary data structure is the SongNode struct. To optimize memory usage, the string fields (title, performer, etc.) are declared as char* pointers rather than fixed-size arrays. Memory is dynamically allocated (malloc) for each string based on its actual length.

```c
typedef struct SongNode {
    char* title;
    char* performer;
    char* album;
    char* genre;
    int release_year;
    int length_min;
    int length_sec;
    struct SongNode* next;
} SongNode;
```

Algorithms

1. **Insertion Sort:** The sorting feature is implemented using **Insertion Sort**. This algorithm was chosen because it is stable, efficient for small datasets, and relatively simple to implement on a linked list by rebuilding a new sorted list node by node.
2. **Dynamic Memory Management:** The program strictly manages memory. Every time a node is created, memory is allocated. Every time a node is deleted (or the program ends), that memory is freed to prevent leaks.

Modules

Although implemented in a single source file, the code is logically divided into:

- **Core Logic:** Managing the linked list (Add, Delete, Free).
- **Sorting Logic:** Handling the insertion sort algorithm.
- **File Handling:** Reading from and writing to text files.
- **User Interface:** Handling menu display and user input.
- **Shortcut Functions:** Handling repetitive tasks (waiting for enter, reading line, comparing strings)

---

2. Function Interfaces

Core Logic Functions

`void add_song_to_list`

- **Description:** Allocates a new node, allocates memory for its strings, populates it with data, and appends it to the end of the list.
- **Input Parameters:**
  - SongNode** root: Double pointer to the start of the list.
  - const char* title, performer...: String data for the song.
  - int year, minutes, seconds: Numeric data for the song.
- **Returned Result:** None (void).

`int delete_song`

- **Description:** Searches for a song matching the specific title and performer. If found, it unlinks the node, frees all its memory, and reconnects the list.
- **Input Parameters:**
  - SongNode** root: Double pointer to the list head.
  - const char* title: Title to match.
  - const char* performer: Performer to match.
- **Returned Result:** 1 if successful, 0 if the song was not found.

`void free_song_list`

- **Description:** Iterates through the entire list and frees every node and every dynamic string within those nodes.

- **Input Parameters:** SongNode** root
- **Returned Result:** None. Sets the root pointer to NULL.

Sorting Functions

`void insertionSort`

- **Description:** The main driver for the sorting logic. It detaches nodes from the original list one by one and inserts them into a new sorted list.
- **Input Parameters:**
  - SongNode** root: Double pointer to the list to be sorted.
  - int sort_type: 1 (Title), 2 (Performer), or 3 (Year).
- **Returned Result:** None. The headRef is updated to point to the new sorted list.

`void sortedInsert`

- **Description:** Helper function that places a single node into the correct position within a sorted list.
- **Input Parameters:**
  - SongNode** sorted_head: The list being built.
  - SongNode* new_node: The node to insert.
  - int sort_type: The sorting criteria.
- **Returned Result:** None.

File Handling Functions

`int load_list_from_file`

- **Description:** Generic function to open any text file, parse songs (7 lines per song), and add them to the in-memory list.
- **Input Parameters:**
  - SongNode** root: List to add songs to.
  - const char* filename: Name of the file to open.
- **Returned Result:** Integer representing the number of songs successfully loaded (or -1 on error).

`int read_line_from_file`

- **Description:** Reads a line from the database file.
- **Input Parameters:**

    - FILE* file: File name to read from.
    - char* buffer: Buffer to store the read data.
    - int size: Size of read data.

- **Returned Result:** 1 if successful, 0 if failed.

`int save_list_to_file`

- **Description:** Writes the current in-memory list to songs.txt in the standard 7-line format.
- **Input Parameters:** SongNode* root
- **Returned Result:** Integer count of songs saved.

User Interface (Menu) Functions

`void print_menu`

- **Description:** Displays the main menu options (1-9) to the console.
- **Input Parameters:** None.
- **Returned Result:** None.

`void do_add_song`

- **Description:** Handles user interaction for adding a song. Prompts the user for details (Title, Artist, etc.), performs input validation (e.g., ensuring positive numbers for year/length), and calls add_song_to_list.
- **Input Parameters:** SongNode** root
- **Returned Result:** None.

`void do_display_sorted`

- **Description:** Prompts the user to select a sorting criteria (Title, Performer, Year), calls insertionSort, and then iterates through the list to print all songs using print_song_details.
- **Input Parameters:** SongNode** root
- **Returned Result:** None.

`void do_list_by_[criteria]`

- **Description:** Set of functions (do_list_by_performer, do_list_by_album, etc.) that prompt the user for a search term, iterate through the list, and print only the nodes that match the criteria.
- **Input Parameters:** SongNode* root
- **Returned Result:** None.

`void do_delete_song`

- **Description:** Prompts the user for a title to delete. Finds matching songs; if multiple matches exist, asks the user to select the specific one. Calls delete_song to perform the actual removal.
- **Input Parameters:** SongNode** root
- **Returned Result:** None.

`void do_import_songs`

- **Description:** Prompts the user for a filename, verifies the file isn't the main database, and calls import_songs_from_file to merge the new songs into the current list.
- **Input Parameters:** SongNode** root
- **Returned Result:** None.

Shortcut Functions

`void read_line`

- **Description:** Reads a line from user input.
- **Input Parameters:**
  - char* buffer: string to save user input to.
  - int size: the size of the buffer needed for `fgets`.
- **Returned Result:** None.

`int stricmp`

- **Description:** Performs case-insensitive comparison between two string one from user input and the other from a string in a node in the linked list.
- **Input Parameters:**
  - const char* s1**:** one of the two strings
  - const char* s2**:** the other string.
- **Returned Result:** an integer representing the result of the comparing (0 if identical).

`void wait_for_enter`

- **Description:** Waits for the user to press “enter” to continue the program.
- **Input Parameters:** None.
- **Returned Result:** None.